CURSO: CC451 — Interacción Humano Computadora

Laboratorio Dirigido: VUI Moderna con LLMs (Groq) + Streamlit

---

1. Introducción

Objetivo general:

Introducción al diseño de interfaces de usuario por voz (VUI) modernas utilizando **Large Language Models (LLMs) de Groq** y **Streamlit**. Implementará un asistente conversacional de pedidos (“SmartPizza VUI”) aplicando los 6 principios de diseño VUI vistos en clase.

Objetivos específicos: - Configurar un pipeline VUI simplificado (ASR simulado → LLM → NLG visual). - Aplicar principios de personalidad, *progressive disclosure*, confirmaciones inteligentes, prevención de errores, memoria conversacional y multimodalidad. - Integrar todo en una interfaz web ejecutable en tiempo real.

---

2. Recursos Informáticos

1. Python 3.9+ y pip.
2. Cuenta gratuita en Groq Cloud → generar API Key.
3. Dependencias Python: `streamlit`, `groq`, `python-dotenv`.
4. Código handout proporcionado (`smartpizza_handout.py`).
5. Documentación de referencia: Groq Docs.

---

3. Desarrollo y Entregable

Al finalizar, comprima en **CC451_LabVUI_<ApellidoNombre>.zip**: - `smartpizza.py` — su código final funcionando. - `screenshots/` — mínimo 3 capturas del flujo conversacional. - `README.txt` — breve explicación de qué principios VUI implementó y por qué.

---

4. Ejercicio — SmartPizza VUI

4a. Configuración Inicial *(~15 min)*

1. Clone/Descargue el handout y el `requirements.txt`.
2. Instale dependencias:

   ```
   pip install -r requirements.txt
   ```

3. Configure su API Key:
   - Opción A (recomendada): crear archivo `.streamlit/secrets.toml` con:

     ```
     GROQ_API_KEY = "gsk_..."
     ```

   - Opción B: exportar como variable de entorno.
4. Ejecute el handout base y verifique que Streamlit levanta sin errores:

   ```
   streamlit run smartpizza_handout.py
   ```

4b. Implementación de Principios VUI *(~90 min)*

Complete los bloques `# TODO` en el handout. Debe implementar **mínimo 4** de los 6 principios:

#	Principio	Qué debe hacer	Archivo/Función
1	**Diseño de Personalidad**	Defina el `SYSTEM_PROMPT` del asistente "Pepper": tono casual, amigable, español neutro/peruano moderado. El LLM nunca debe inventar productos fuera del menú.	`smartpizza.py` — constante `SYSTEM_PROMPT`
2	**Progressive Disclosure**	En la bienvenida (`main()`) muestre solo **2-3 opciones** iniciales (ej: *Hacer pedido*, *Ver promociones*, *Reordenar último*). Revelar sub-	`main()` — bloque de bienvenida

#	Principio	Qué debe hacer	Archivo/Función
		opciones solo después de la elección.	
3	**Confirmaciones Inteligentes**	Implemente `check_confirmation_needed(text)`: si detecta palabras de alto riesgo (*cancela*, *borra*, *modifica*), exija confirmación explícita. Si es bajo riesgo (*agrega*, *quiero*), use confirmación implícita.	`check_confirmation_needed()`
4	**Prevención de Errores**	Implemente `validate_order_input(text)` para rechazar ítems fuera del menú (ej: *hamburguesa*, *sushi*). Use clarificación proactiva para tamaños. Implemente *defaults inteligentes* (última dirección conocida).	`validate_order_input()`
5	**Memoria Conversacional**	Complete `get_llm_response()` para enviar al LLM los **últimos 5 turnos** de la conversación como contexto. Extraiga y guarde el nombre del	`get_llm_response()` + session state

#	Principio	Qué debe hacer	Archivo/Función
		usuario si dice *“me llamo …”*.	
6	**Multimodalidad Inteligente**	Complete `render_multimodal_menu()` usando `st.columns()` y `st.metric()` para mostrar pizzas/precios visualmente. Agregue botones rápidos debajo del chat.	`render_multimodal_menu()` + botones

4c. Integración y Pruebas *(~15 min)*

1. Integre todas las funciones en `main()`.
2. Pruebe el flujo completo:
   - Bienvenida progresiva → selección de pizza → clarificación de tamaño → confirmación → despedida.
3. Pruebe un *error* (pedir sushi) y verifique que el sistema corrige amablemente.
4. Capture 3 screenshots del flujo.

---

5. Tarea — Extensión Obligatoria

Además del ejercicio base, agregue las siguientes mejoras:

- **Persistencia de memoria:** Use `sqlite3` para guardar el perfil del usuario y su historial de pedidos entre sesiones (memoria persistente, no solo de sesión).
- **Flujo de recompensas:** Después de detectar el segundo pedido en la conversación, ofrezca un descuento especial (ej: *“¡Llevas 2 pedidos! Tienes 20% de descuento en bebidas”*).

- **Simulación de estados VUI:** Agregue indicadores visuales animados en Streamlit (`st.spinner`, `st.toast`, emojis dinámicos 🔊🎙️) que simulen los estados *“Escuchando”*, *“Procesando”* y *“Hablando”* de un pipeline de voz real.

Entregue: ZIP con código, screenshots y README explicando la extensión elegida.
